

Gabarito Completo: Aula 21 - Arrays e Objetos

Este documento contém a resolução detalhada dos 20 exercícios propostos na aula prática sobre manipulação de coleções de dados e JSON em JavaScript.

Nível 1: Fundamentos de Objetos e Arrays (Básico)

1. Criação de Registro Único

Exercício: Crie um objeto chamado **produto** que represente um item de estoque. Ele deve conter as propriedades: **id** (número), **nome** (string), **preco** (número) e **emEstoque** (booleano).

```
const produto = {
```

```
  id: 1,
```

```
  nome: "Teclado Mecânico",
```

```
  preco: 250.00,
```

```
  emEstoque: true
```

```
};
```

2. Acesso Dinâmico

Exercício: Utilizando o objeto criado, acesse o nome do produto usando a notação de ponto e o preço usando a notação de colchetes.

```
console.log(produto.nome); // Saída: Teclado Mecânico
```

```
console.log(produto["preco"]); // Saída: 250.00
```

3. Expansão de Objeto

Exercício: Adicione dinamicamente uma propriedade chamada **categoria** ao objeto **produto** e atribua a ela um valor.

```
produto.categoria = "Periféricos";
```

4. Lista de Registros

Exercício: Crie um array chamado **inventario** contendo três objetos de produtos com estruturas semelhantes.

```
const inventario = [
```

```
  { id: 1, nome: "Teclado", preco: 250, emEstoque: true, categoria: "Periféricos" },
```

```
  { id: 2, nome: "Mouse Gamer", preco: 150, emEstoque: true, categoria: "Periféricos" },
```

```
{ id: 3, nome: "Monitor 144hz", preco: 1200, emEstoque: false, categoria: "Eletrônicos" }
```

```
];
```

5. Contagem de Itens

Exercício: Acesse a propriedade `length` do seu array `inventario` e exiba quantos produtos estão registrados.

```
console.log(`Total de produtos registrados: ${inventario.length}`);
```

Nível 2: Manipulação e Iteração Básica (Intermediário)

6. Adição Manual

Exercício: Use o método `push()` para adicionar um novo cliente (objeto com nome e email) a um array chamado `listaClientes`.

```
const listaClientes = [];
```

```
listaClientes.push({ nome: "João Silva", email: "joao@email.com" });
```

7. Varredura com `forEach`

Exercício: Utilize o método `forEach()` para percorrer o array `inventario` e exibir no console apenas o nome de cada produto.

```
inventario.forEach(produto => {
```

```
    console.log(produto.nome);
```

```
});
```

8. Remoção de Registro

Exercício: Utilize o método `splice()` para remover o segundo produto da sua lista de `inventario`.

```
// Remove 1 item a partir do índice 1 (segundo elemento)
```

```
inventario.splice(1, 1);
```

9. Busca Simples

Exercício: Crie um array de strings com nomes de categorias. Use `indexOf()` para verificar a existência de uma categoria.

```
const categorias = ["Periféricos", "Eletrônicos", "Escritório"];
```

```
const index = categorias.indexOf("Eletrônicos");
```

```
const existe = index !== -1;
```

```
console.log(`A categoria existe? ${existe}`);
```

10. Cópia de Referência vs. Valor

Exercício: Demonstre como a alteração em uma variável que aponta para um objeto afeta o original (referência).

```
let clienteOriginal = { nome: "Ana" };
```

```
let clientePremium = clienteOriginal; // Ambos apontam para o mesmo local na memória
```

```
clienteOriginal.nome = "Ana Souza";
```

```
console.log(clientePremium.nome); // Saída: "Ana Souza"
```

Nível 3: Métodos Funcionais de Array (Avançado)

11. Filtragem de Ativos (Filter)

Exercício: Utilize o método `filter()` para retornar um novo array contendo apenas os agendamentos com status: "pendente".

```
const agendamentos = [
```

```
  { data: "2026-04-16", status: "pendente" },
```

```
  { data: "2026-04-17", status: "pago" },
```

```
  { data: "2026-04-18", status: "pendente" }
```

```
];
```

```
const pendentes = agendamentos.filter(a => a.status === "pendente");
```

12. Transformação de Preços (Map)

Exercício: Utilize o método `map()` no array `inventario` para criar um novo array que contenha os preços de todos os produtos com um acréscimo de 10% (impostos).

```
const precosComImposto = inventario.map(p => p.preco * 1.10);
```

13. Resumo de Estoque (Reduce)

Exercício: Utilize o método `reduce()` para calcular o valor total financeiro de todos os itens do `inventario` somados.

```
const valorTotal = inventario.reduce((acumulador, p) => acumulador + p.preco, 0);
```

14. Verificação de Regra de Negócio (Every)

Exercício: Use o método `every()` para verificar se todos os produtos do inventário possuem um preço superior a zero.

```
const todosValidos = inventario.every(p => p.preco > 0);
```

15. Busca por Critério (Some)

Exercício: Use o método `some()` para verificar se existe algum produto no inventário que pertença à categoria "Promoção".

```
const temPromocao = inventario.some(p => p.categoria === "Promoção");
```

Nível 4: JSON e Lógica de Integração (Expert)

16. Serialização de Dados

Exercício: Converta seu array de `inventario` (objetos) em uma string no formato JSON utilizando `JSON.stringify()`.

```
const jsonInventario = JSON.stringify(inventario);
```

17. Desserialização de API

Exercício: Receba uma string JSON simulando uma resposta de servidor e converta-a de volta para um objeto JavaScript usando `JSON.parse()`.

```
const jsonResposta = '{"id":1, "status":"pago"}';
```

```
const objetoResposta = JSON.parse(jsonResposta);
```

18. Ordenação Técnica

Exercício: Utilize o método `sort()` para ordenar o array `inventario` em ordem decrescente de preço.

```
inventario.sort((a, b) => b.preco - a.preco);
```

19. Extração de Chaves

Exercício: Use o método `Object.keys()` para listar todos os nomes de propriedades que um objeto de agendamento possui.

```
const chaves = Object.keys(agendamentos[0]);
```

```
console.log(chaves); // Ex: ["data", "status"]
```

20. Clonagem Estruturada

Exercício: Crie uma função que receba um array de objetos de clientes e retorne uma cópia profunda (deep copy).

```
function copiarProfundo(lista) {  
  
    return JSON.parse(JSON.stringify(lista));  
  
}  
  
const novaListaClientes = copiarProfundo(listaClientes);
```